BITS Pilani - Machine Learning Operations (MLOps) AIMLCZG523

Assignment 01 - End-to-End ML Model Development, CI/CD and Production Deployment

Heart Disease Risk Prediction as a Containerised, Monitored API

Student	Sarth
Dataset	UCI Heart Disease (Cleveland), 14 attributes, binary target
Stack	scikit-learn, MLflow, Flask + gunicorn, Docker, Kubernetes, Prometheus, Grafana, GitHub Actions
Repository	https://github.com//heart-disease-mlops
Report generated from	reports/metrics.json (data source: UCI processed.cleveland.data via HTTPS)

Contents

- 1. Introduction and objectives
- 2. Repository layout and setup instructions
- 3. Data acquisition and cleaning
- 4. Exploratory data analysis
- 5. Feature engineering and model development
- 6. Experiment tracking (MLflow)
- 7. Model packaging and reproducibility
- 8. Automated testing and CI/CD pipeline
- 9. Containerisation (Docker)
- 10. Production deployment (Kubernetes)
- 11. Monitoring and logging
- 12. Architecture overview
- 13. Results, limitations and ethical considerations
- 14. Requirement-to-evidence checklist
- Appendix A - API reference, Appendix B - commands

Marks map: EDA 5, modelling 8, tracking 5, packaging 7, CI/CD 8, containerisation 5, deployment 7, monitoring 3, documentation 2 (total 50).

1. Introduction and objectives

Cardiovascular disease is a leading cause of death worldwide, and early risk stratification from routine clinical measurements can prioritise patients for further testing. This project builds a binary classifier that estimates the risk of heart disease from 13 patient attributes (age, sex, chest-pain type, blood pressure, cholesterol, ECG findings, exercise test results and fluoroscopy findings) and ships it the way a production ML system should be shipped: reproducible, tested, tracked, containerised, deployed to Kubernetes and monitored.

Design goals

- **Reproducibility** - one command per stage, pinned dependencies, fixed seeds, data hash and library versions stored with the model.
- **No train/serve skew** - a single scikit-learn Pipeline (feature engineering + imputation + scaling + encoding + classifier) is trained, cross-validated, pickled and served.
- **No data leakage** - imputation and scaling are fitted inside cross-validation folds only.
- **Fail loudly** - schema validation on data and on API requests, tests gate CI, the pipeline stops on any error.
- **Observability** - structured JSON logs, request IDs, Prometheus metrics, Grafana dashboard and alert rules.
- **Privacy by default** - patient feature values are never logged unless explicitly enabled.

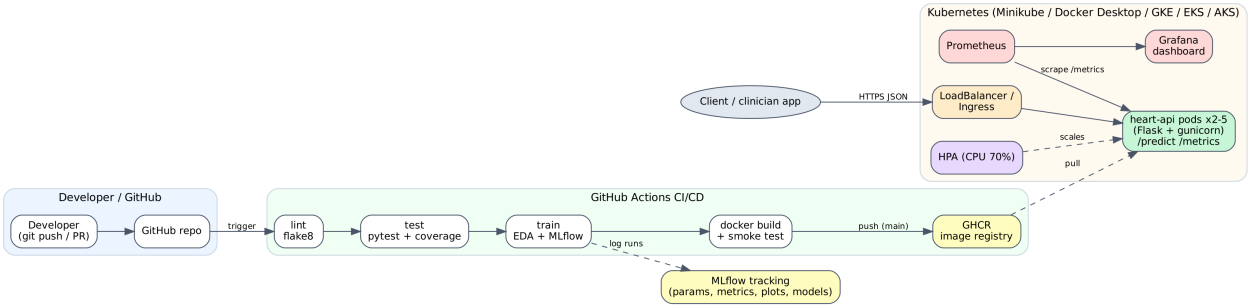


Figure 1. End-to-end system architecture: GitHub, CI/CD, MLflow, container registry, Kubernetes serving and monitoring.

2. Repository layout and setup instructions

```
heart-disease-mlops/
  heartml/            package: config, data, features, modeling, tracking, train, eda, plots, predict, schema
  app/                Flask API (main.py) + dependency-free Prometheus registry (metrics.py)
  scripts/            download_data.py, smoke_test.py, load_test.py
  tests/              48 unit tests (data, features, model/training/tracking, API, metrics)
  notebooks/          01_eda, 02_training, 03_inference
  data/raw|processed  downloaded dataset and cleaned dataset (+ DATA_SOURCE.txt provenance marker)
  models/             model.joblib + model_metadata.json
  reports/            metrics.json, eda_summary.json, figures/, build_report.py
  docs/diagrams/      architecture diagrams (+ build script);  docs/evidence/  captured local run outputs
  k8s/                namespace, configmap, deployment, services, ingress, HPA, Prometheus, Grafana, kustomization
  monitoring/         prometheus.yml, alerts.yml, Grafana provisioning + dashboard JSON (docker-compose stack)
  .github/workflows/  ci.yml        Dockerfile  docker-compose.yml  Makefile  requirements*.txt
```

Quick start (clean machine)

```
git clone <repo-url> && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate        # Python 3.13
pip install -r requirements-dev.txt

python scripts/download_data.py        # real UCI data (ucimlrepo, falls back to direct HTTPS)
make all                               # clean -> EDA -> train (3 models, MLflow tracked)
make test                              # 48 unit tests with coverage
make run                               # API on http://localhost:8000

docker build -t heart-disease-api:1.0.0 .
```

```
docker run --rm -p 8000:8000 heart-disease-api:1.0.0
python scripts/smoke_test.py http://localhost:8000

docker compose up --build              # API + Prometheus + Grafana
kubectl apply -k k8s/                  # Kubernetes (Minikube / Docker Desktop / cloud)
```

The download script tries the `ucimlrepo` package (dataset id 45), then a direct HTTPS download of `processed.cleveland.data`. If both fail it prints manual instructions. An explicit `--synthetic` flag exists only for offline dry runs; it writes a provenance marker that is propagated to the logs, the tracker, `model_metadata.json`, `/model-info` and this report so synthetic results can never be mistaken for real ones.

3. Data acquisition and cleaning

The Cleveland subset of the UCI Heart Disease database has 303 patients and 13 predictors. The original target (*num*) grades disease severity 0-4; following common practice it is binarised: 0 = no disease, 1-4 = disease present. Missing values are encoded as '?' in the raw file.

Feature	Type	Description
age	numeric	Age in years
sex	categorical	Sex (1 = male, 0 = female)
cp	categorical	Chest pain type (1 typical angina, 2 atypical, 3 non-anginal, 4 asymptomatic)
trestbps	numeric	Resting blood pressure (mm Hg)
chol	numeric	Serum cholesterol (mg/dl)
fbs	categorical	Fasting blood sugar > 120 mg/dl (1 = true)
restecg	categorical	Resting ECG (0 normal, 1 ST-T abnormality, 2 LV hypertrophy)
thalach	numeric	Maximum heart rate achieved
exang	categorical	Exercise-induced angina (1 = yes)
oldpeak	numeric	ST depression induced by exercise relative to rest
slope	categorical	Slope of peak exercise ST segment (1 up, 2 flat, 3 down)
ca	numeric	Number of major vessels (0-3) coloured by fluoroscopy
thal	categorical	Thalassemia (3 normal, 6 fixed defect, 7 reversible defect)

Table 1. Input features.

Cleaning steps (`heartml/data.py::clean`)

- Schema validation - fails with *DataValidationError* if any expected column is absent; columns are re-ordered deterministically.
- Coercion - every column is converted to numeric; '?' and any unparseable token become NaN.
- Plausibility rules - physiologically impossible values (e.g. cholesterol 0, systolic BP 900, undefined category codes) are set to NaN and logged.
- Duplicate rows are dropped; rows without a target are removed; target is binarised.
- **Missing values are kept as NaN** in the cleaned CSV. Imputation (median for numerics, most-frequent for categoricals) is a pipeline step fitted on training folds only, which prevents leakage and guarantees that the API handles missing values exactly as training did.

After cleaning the dataset has **303** rows; missing values: **ca** = 4, **thal** = 2.

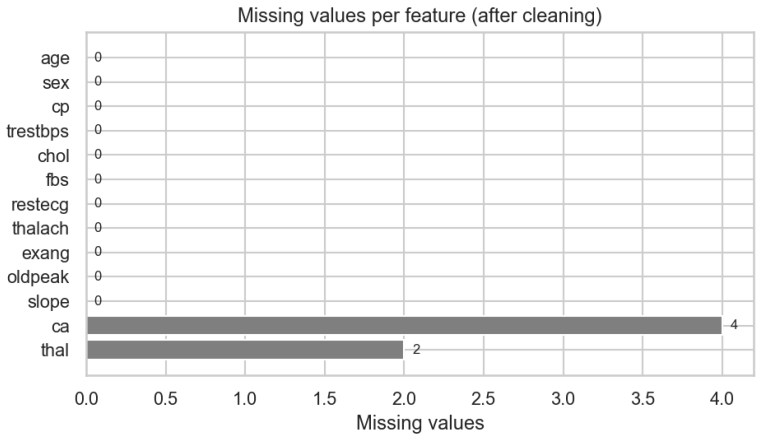


Figure 2. Missing values per feature after cleaning.

4. Exploratory data analysis

Class balance. 164 patients without disease and 139 with disease (45.9% positive). The classes are nearly balanced, so accuracy is a meaningful metric, but because a missed case is more costly than a false alarm, recall and ROC-AUC are monitored as well and used for model selection tie-breaking.

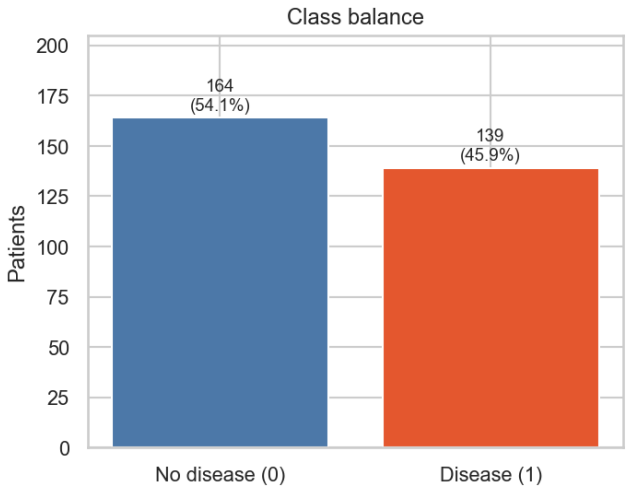


Figure 3. Class balance.

Distributions. Histograms with kernel density estimates, split by class, show which numeric features separate the classes.

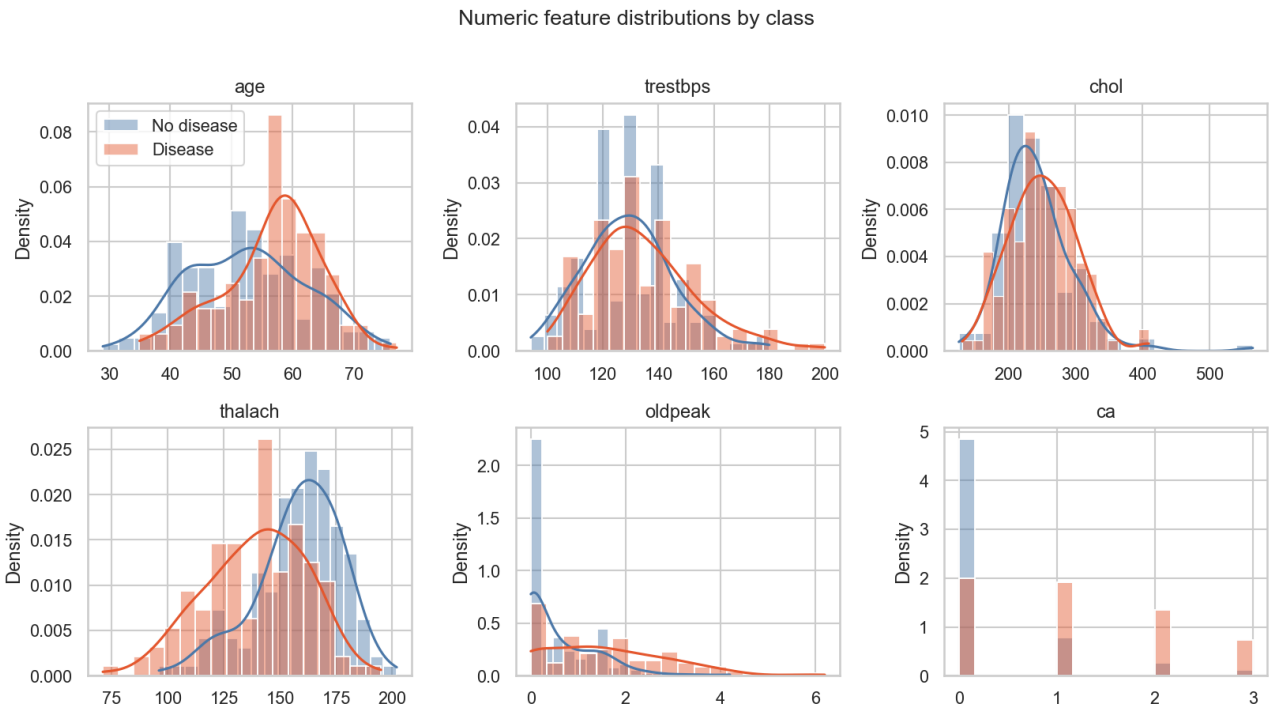


Figure 4. Numeric feature distributions by class.

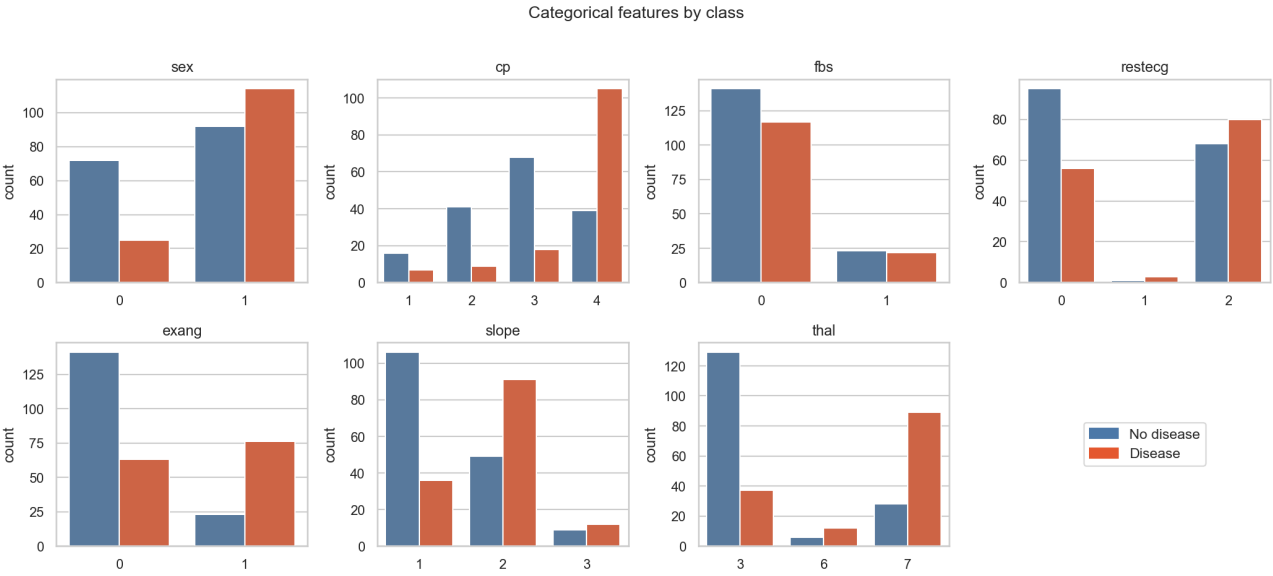


Figure 5. Categorical feature counts by class.

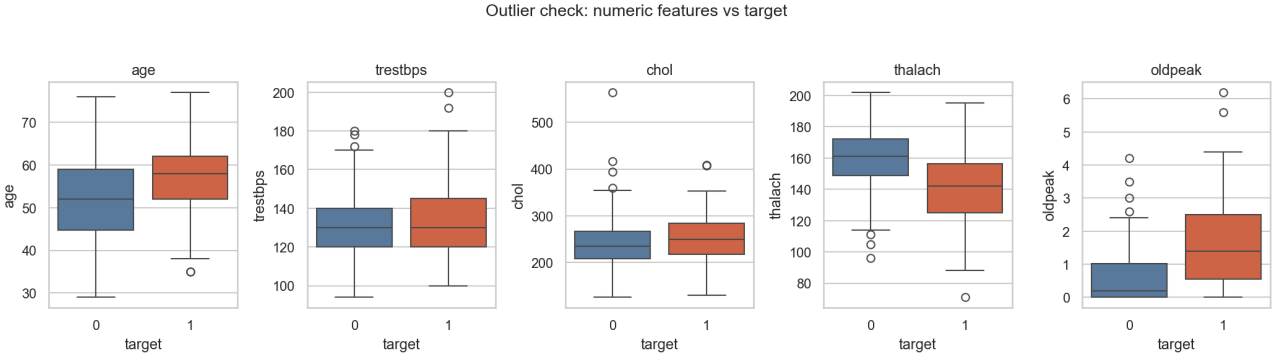


Figure 6. Box plots for outlier inspection. Extreme but plausible values are retained; only impossible values are nulled.

Correlation. Pearson correlations with the target: strongest positive **thal** (+0.53), **ca** (+0.46), **exang** (+0.43); strongest negative **fbs** (+0.03), **thalach** (-0.42). No pair of predictors is so collinear that it threatens the linear model; regularisation handles the moderate correlations.

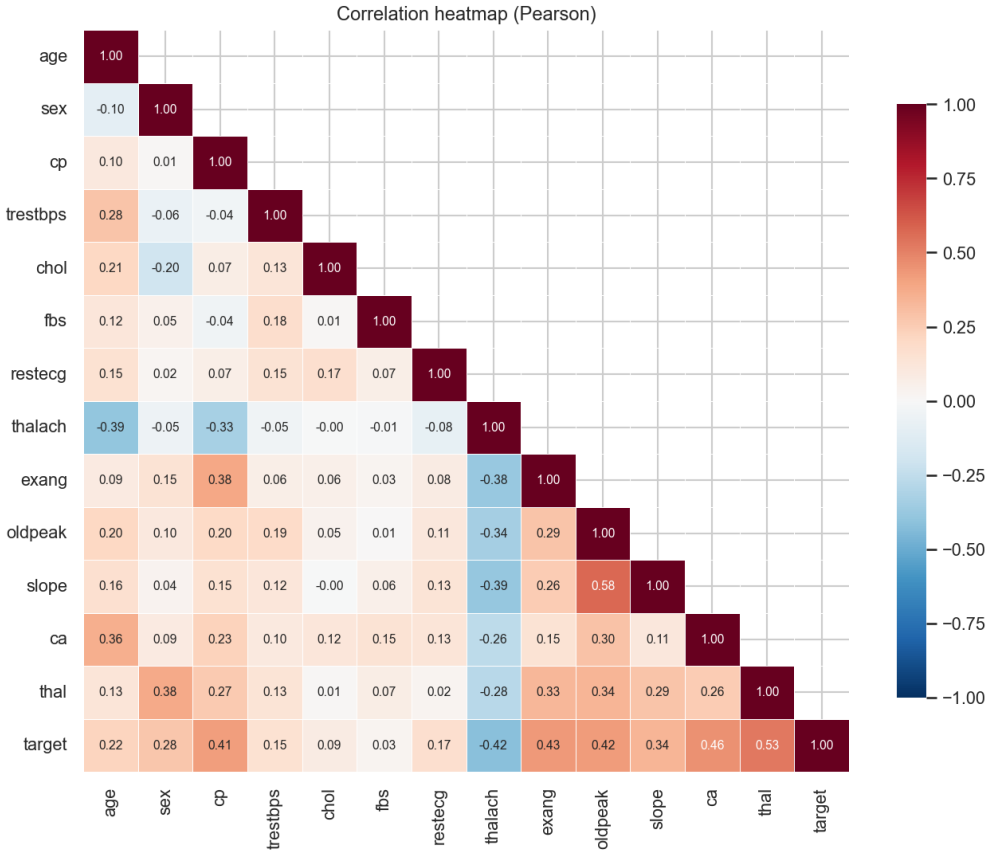


Figure 7. Correlation heatmap (lower triangle).

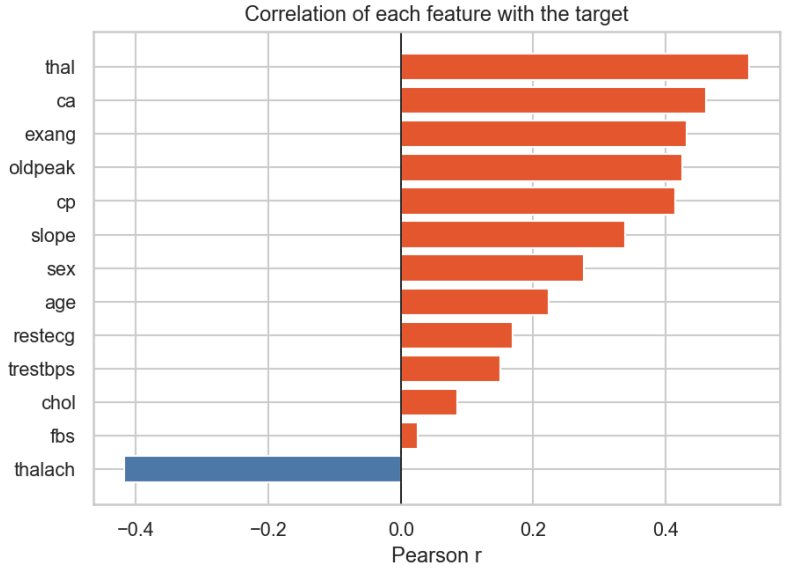


Figure 8. Correlation of each feature with the target.

Subgroup rates. Disease prevalence by sex: female 25.8%, male 55.3%; with exercise-induced angina 76.8% vs 30.9% without. Males make up 68% of the sample, which is a representativeness limitation discussed in Section 13.

5. Feature engineering and model development

Preprocessing pipeline. A scikit-learn *ColumnTransformer* inside a single *Pipeline*:

Step	Applies to	Operation
FeatureAdder	all rows	hr_reserve_ratio = thalach / (220 - age); chol_per_age = chol / age; high_bp = (trestbps >= 140)
Numeric branch	age, trestbps, chol, thalach, oldpeak, ca + 2 engineered	median imputation -> StandardScaler
Categorical branch	sex, cp, fbs, restecg, exang, slope, thal + high_bp	most-frequent imputation -> OneHotEncoder(handle_unknown='ignore')
Classifier	-	Logistic Regression / Random Forest / Gradient Boosting

Table 2. Pipeline stages.

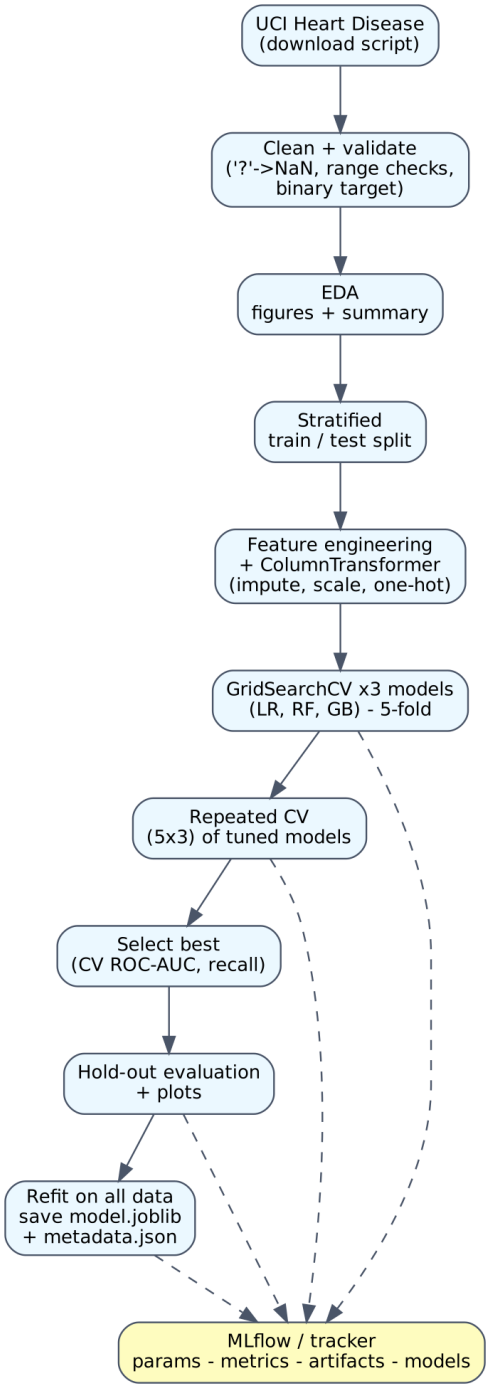


Figure 9. Training pipeline from raw data to a saved, tracked model.

The engineered features encode clinical knowledge: the heart-rate reserve ratio compares the achieved maximum heart rate with the age-predicted maximum, cholesterol is normalised by age, and a blood-pressure flag marks the stage-2 hypertension threshold.

Models and tuning. Three classifiers with different inductive biases were compared. Each was tuned with an exhaustive grid search using 5-fold stratified cross-validation on the training split (80 %), optimising ROC-AUC. The tuned models were then re-evaluated with **repeated stratified 5-fold CV (3 repeats = 15 fits)** to obtain stable mean +/- std estimates, and finally scored once on the 20 % hold-out test set.

Model	Search space	Grid size	Best parameters
Logistic Regression	C in {0.01..10}, class_weight in {None, balanced}	14	C=0.1, class_weight=balanced
Random Forest	n_estimators {150,300}, max_depth {3,5,8,None}, min_samples_leaf {1,3,5}	24	max_depth=None, min_samples_leaf=5, n_estimators=150
Gradient Boosting	n_estimators {50,100,200}, learning_rate {0.03,0.1}, max_depth {2,3}	12	learning_rate=0.03, max_depth=2, n_estimators=100

Table 3. Hyper-parameter search and selected values.

Cross-validated results (repeated stratified 5-fold, mean ± std):

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression *	0.829 ± 0.056	0.835 ± 0.077	0.787 ± 0.073	0.808 ± 0.064	0.895 ± 0.041
Random Forest	0.797 ± 0.054	0.811 ± 0.087	0.738 ± 0.078	0.769 ± 0.061	0.887 ± 0.041
Gradient Boosting	0.797 ± 0.053	0.818 ± 0.089	0.730 ± 0.069	0.768 ± 0.058	0.873 ± 0.043

Table 4. Cross-validation metrics (* = selected model).

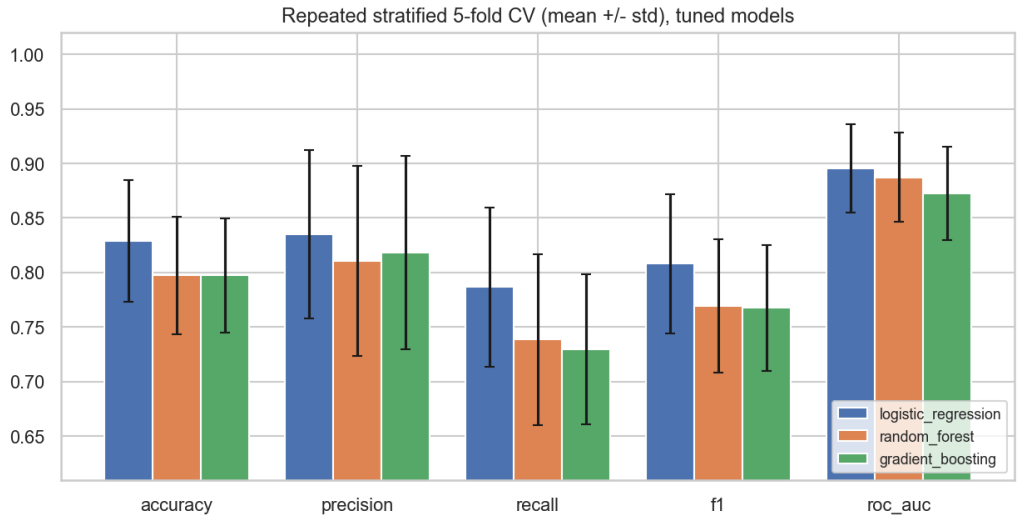


Figure 10. Cross-validated metrics of the three tuned models (error bars = std).

Hold-out test results (61 patients, threshold 0.5):

Model	Accuracy	Precision	Recall	Specificity	F1	ROC-AUC	Brier
Logistic Regression *	0.885	0.839	0.929	0.848	0.881	0.960	0.091
Random Forest	0.852	0.788	0.929	0.788	0.852	0.961	0.102
Gradient Boosting	0.885	0.862	0.893	0.879	0.877	0.950	0.101

Table 5. Hold-out test metrics.

Model selection. Logistic Regression was selected: highest cross-validated ROC-AUC (0.895), ties broken by recall because a false negative (missed disease) is the costlier error. On the hold-out set it produced 26 true positives, 28 true negatives, 5 false positives and 2 false negatives. Selection used cross-validation only - the test set was never used to choose a model, so its scores are an unbiased estimate.

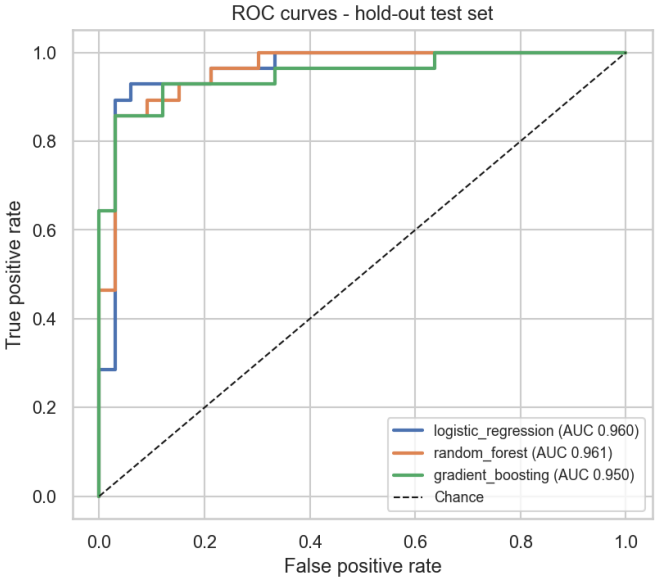


Figure 11. ROC curves on the hold-out test set.

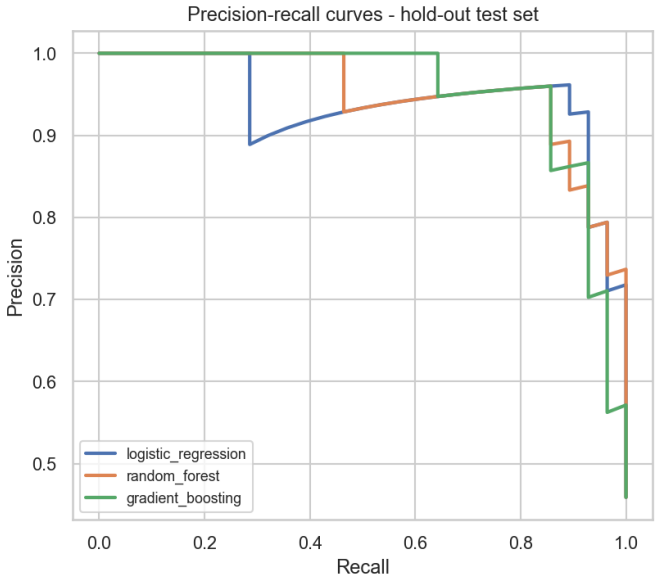


Figure 12. Precision-recall curves on the hold-out test set.

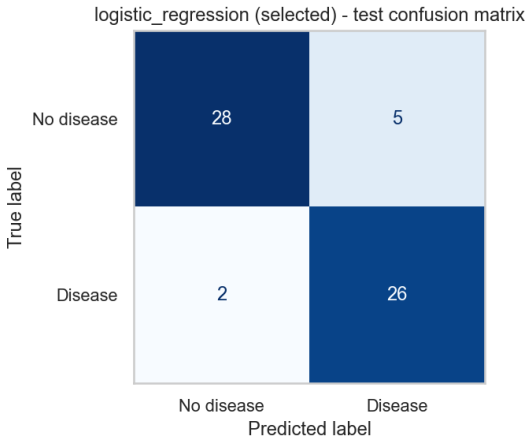


Figure 13. Confusion matrix of the selected model.

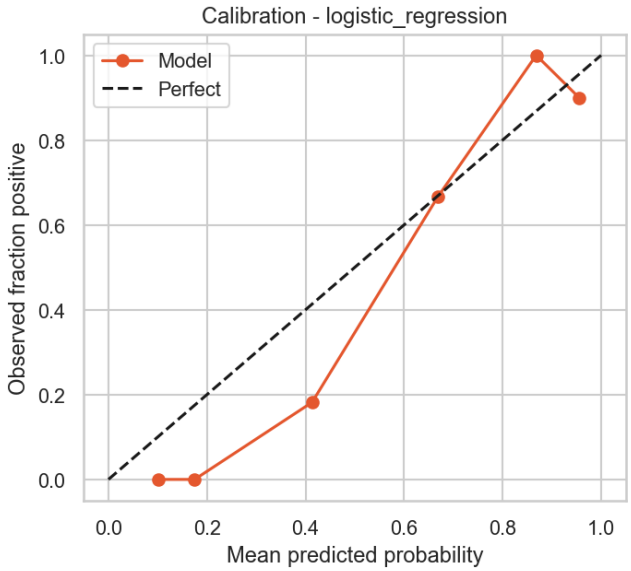


Figure 14. Calibration curve of the selected model (reliability diagram). Probabilities reported by the API are used as the confidence value.

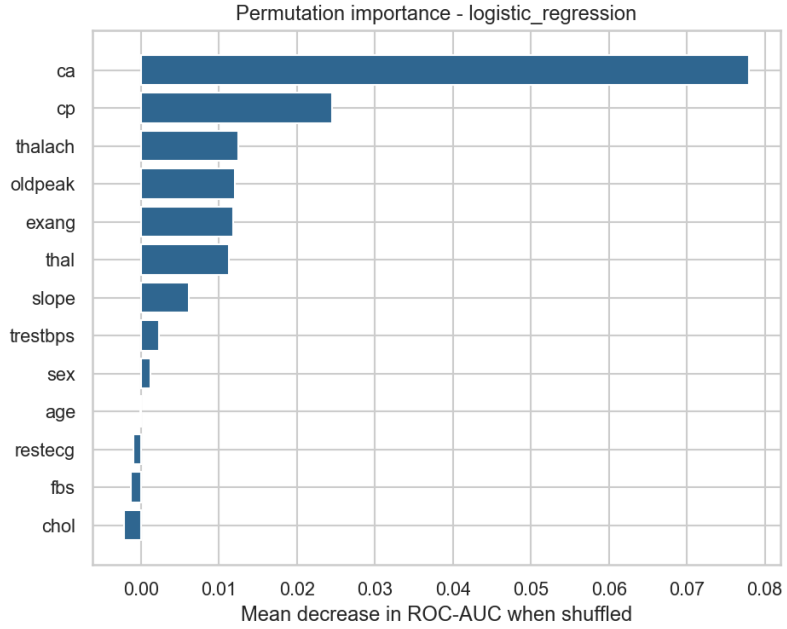


Figure 15. Permutation importance of the raw input features on the test set (drop in ROC-AUC when shuffled).

The most influential inputs were **ca** (0.078), **cp** (0.024), **thalach** (0.012), **oldpeak** (0.012).

Notes and caveats. The final deployed model is refit on *all* rows with the CV-selected hyper-parameters, a standard practice once model selection is complete; its hold-out score therefore is not independent and the figures above (from the train-only fit) are the ones to quote. With only 303 rows, confidence intervals are wide (see the std columns), so differences of 1-2 points between models are not significant. If a selected hyper-parameter sits at the edge of its grid (e.g. the largest C), a wider grid should be explored.

6. Experiment tracking (MLflow)

Every run is tracked through `heartml/tracking.py`, which uses **MLflow** when installed (the default via requirements.txt) and a small file-based fallback with the same API when it is not, so the pipeline never silently loses tracking. The structure is a **parent run** (*training-pipeline*) with one **nested child run per model**.

Logged item	Parent run	Child run (per model)
Parameters	rows, split sizes, seed, CV folds/repeats, data SHA-256, data source, library versions, models compared, selected model + params	model name, every best_* hyper-parameter, grid size, search metric
Metrics	selected_test_* (accuracy, precision, recall, specificity, F1, ROC-AUC, AP, Brier), selected_cv_*	cv_*_mean / cv_*_std for 5 metrics, grid_best_cv_roc_auc, test_* metrics
Artifacts / plots	ROC, PR, comparison, calibration, confusion, importance, final model + metadata, metrics.json	confusion matrix, full grid-search table (CSV)
Models	final pipeline (MLflow sklearn flavour, with input example)	tuned pipeline per model
Tags	data_source, git_sha	model

Table 6. What is logged to MLflow.

Tracking backend used to generate this report: **local** (`heart-disease-risk`). Browse runs with `mlflow ui --backend-store-uri ./mlruns` (or `make mlflow-ui`) at http://localhost:5000.

SCREENSHOT TO ADD: `screenshots/mlflow_runs_list.png`
MLflow UI - experiment 'heart-disease-risk' with the parent run and three nested model runs.

SCREENSHOT TO ADD: `screenshots/mlflow_run_detail.png`
MLflow UI - run detail with parameters, metrics and logged plots/artifacts.

SCREENSHOT TO ADD: `screenshots/mlflow_compare.png`
MLflow UI - comparison of the three model runs.

A summary of the tuning process is saved for every model as `reports/figures/cv_results_<model>.csv` (all grid points ranked by CV ROC-AUC), making the selection auditable.

7. Model packaging and reproducibility

- **Reusable format.** `models/model.joblib` is the whole fitted Pipeline (feature engineering + preprocessing + classifier), so serving needs no hand-written preprocessing. The same pipeline is also logged to MLflow's sklearn flavour.
- **Metadata.** `models/model_metadata.json` records model name and version, UTC training time, best parameters, hold-out and CV metrics, decision threshold, data source and SHA-256, git commit, and the exact Python / scikit-learn / NumPy / pandas versions.
- **Pinned environments.** `requirements-api.txt` (serving, small) and `requirements.txt` / `requirements-dev.txt` (full / dev). Serving pins must match the training versions because pickles are version-sensitive; the container image installs exactly those pins.
- **Determinism.** Fixed random seeds for the split, CV folds, and models; a unit test asserts that tuning twice gives identical results.

- **Clean-setup execution.** Every stage is a single command (Makefile) that works from a fresh virtual environment: download -> clean -> EDA -> train -> test -> serve.

Field	Value
model_name	logistic_regression
model_version	1.0.0-20261020171437
trained_at_utc	2026-10-20T11:44:37Z
data_source	UCI processed.cleveland.data via HTTPS
sklearn_version	1.8.0
python_version	3.13.14
numpy_version	2.4.3
pandas_version	3.0.1

Table 7. Excerpt of model_metadata.json stored beside the model.

8. Automated testing and CI/CD pipeline

Unit tests (`tests/`, pytest-compatible, written with unittest classes): **48 tests** in four modules.

Module	Covers
test_data.py	'?' to NaN, target binarisation, impossible-value nulling, duplicate removal, schema errors, column ordering, stratified and reproducible splits
test_features.py	engineered feature values, input not mutated, NaN propagation, type check, no NaNs reach the classifier, scaling and one-hot output, unseen categories, imputation at predict time
test_model.py	metric ranges and perfect-score case, model selection rule, a full training run on small data: artefacts written, tracking params/metrics/plots logged, probabilities valid, determinism
test_api.py	request schema (missing/unknown/type/range/null), Prometheus renderer + label escaping, every endpoint, 200/400/404/405/413/422 paths, batch input, metrics counters

Table 8. Test suite.

Training tests use tiny search grids so the whole suite finishes in about 12 seconds. Captured output of the last local run:

```
----------------------------------------------------------------------
Ran 48 tests in 12.007s

OK
```

GitHub Actions workflow (`.github/workflows/ci.yml`) runs on every push and pull request to *main*:

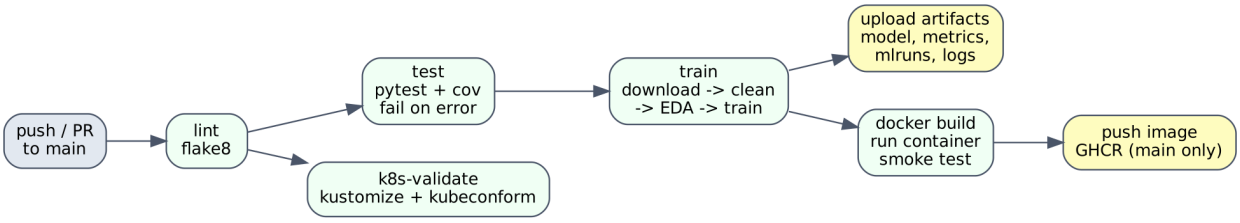


Figure 16. CI/CD workflow. Each job depends on the previous one, so any failure stops the pipeline.

Job	Steps	Artifacts / logs
lint	flake8 over the repo (fails on any violation)	annotated source lines in the log
test	pip install -r requirements-dev.txt; pytest with coverage	test-results.xml, coverage.xml
train	download UCI data -> clean -> EDA -> train (MLflow); job summary table	models/, reports/, mlruns/, train.log
docker	download the model, docker build, run container, run smoke test against it; push to GHCR on main only	container.log
k8s-validate	kustomize render + kubeconform strict schema validation	log

Table 9. Pipeline jobs.

The pipeline **fails on code or test errors** with clear logs: jobs are chained with `needs:`, the training step uses `set -o pipefail` so a crash is not hidden by `tee`, and the smoke test exits non-zero on the first failed check.

> **SCREENSHOT TO ADD:** `screenshots/github_actions_run.png`
> GitHub Actions - successful run showing all jobs (lint, test, train, docker, k8s-validate).

SCREENSHOT TO ADD: `screenshots/github_actions_artifacts.png`
GitHub Actions - uploaded artifacts (test reports, training outputs, container logs).

SCREENSHOT TO ADD: `screenshots/github_actions_failure_demo.png`
Optional: a deliberately failing run (e.g. a broken test) showing the pipeline stops with a clear error.

9. Containerisation (Docker)

The API is a Flask application served by gunicorn. The image is based on *python:3.13-slim*, installs only the small serving requirements, copies the code and trained model, runs as an **unprivileged user** (uid 10001) and defines a **HEALTHCHECK**. gunicorn runs **one worker with four threads** so the in-process Prometheus registry sees every request; horizontal scaling is done with Kubernetes replicas.

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 PORT=8000 MODEL_PATH=/app/models/model.joblib
WORKDIR /app
COPY requirements-api.txt .
RUN pip install --no-cache-dir -r requirements-api.txt     # cached layer
COPY heartml/ heartml/   COPY app/ app/   COPY models/ models/
RUN useradd --create-home --uid 10001 appuser && chown -R appuser /app
USER appuser
HEALTHCHECK ... GET /health
CMD ["gunicorn","--bind","0.0.0.0:8000","--workers","1","--threads","4","app.main:create_app()"]
```

Endpoints. `POST /predict` accepts one JSON object (or a list / *{"instances": [...]}*, max 100) and returns the prediction and confidence; `GET /health`, `/ready`, `/model-info` and `/metrics` support operations.

Sample request and response (captured from the locally running API):

```
$ curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d '{"age":63,"sex":1,"cp":1,"trestbps":145,
    "chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
{"prediction":1,"label":"heart disease predicted","probability_disease":0.7486,"confidence":0.7486,"model_version":"1.0.
```

Invalid input is rejected with HTTP 422 and a precise list of problems (captured):

```
{
 "error": "validation failed",
 "details": [
  {
   "index": 0,
   "errors": [
    "age=999 outside allowed range [1, 120]",
    "missing field: sex",
    "missing field: cp",
    "..."
   ]
  }
 ]
}
```

Local smoke test (`scripts/smoke_test.py`, run against the live server; the same script gates the Docker CI job):

```
[PASS] GET /health {"status":"ok"}
[PASS] GET /ready
[PASS] POST /predict (valid) {"prediction":1,"label":"heart disease predicted","probability_disease":0.7486,"confidence"
[PASS] POST /predict (invalid -> 422)
[PASS] POST /predict (batch)
[PASS] GET /metrics
All smoke tests passed.
```

Docker proof required by the assignment - to be captured on a machine with Docker:

> **SCREENSHOT TO ADD:** `screenshots/docker_build.png`
> Terminal - docker build -t heart-disease-api:1.0.0 . completing successfully.

> **SCREENSHOT TO ADD:** `screenshots/docker_run_predict.png`
> Terminal - docker run -p 8000:8000 and a successful curl /predict against the container.

> **SCREENSHOT TO ADD:** `screenshots/docker_smoke_test.png`
> Terminal - python scripts/smoke_test.py http://localhost:8000 against the container (all PASS).

10. Production deployment (Kubernetes)

Manifests in `k8s/` are applied with `kubectl apply -k k8s/` (kustomize). They run unchanged on Minikube, Docker Desktop Kubernetes, GKE, EKS or AKS; only the image reference and the Service type behaviour differ (see below).

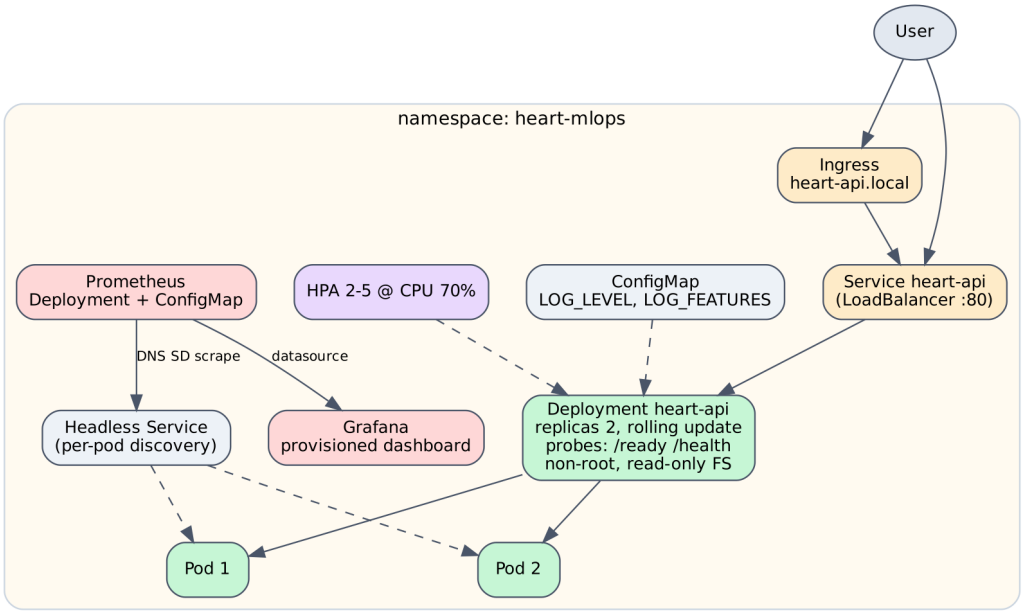


Figure 17. Kubernetes resources in the heart-mlops namespace.

Resource	Purpose / key settings
Namespace heart-mlops	isolates all resources
ConfigMap heart-api-config	LOG_LEVEL, LOG_FEATURES=false, MODEL_PATH - configuration separated from the image
Deployment heart-api	2 replicas, rolling update (maxUnavailable 0), readiness /ready, liveness /health, CPU/memory requests+limits, runAsNonRoot, read-only root FS, all capabilities dropped, seccomp RuntimeDefault
Service heart-api (LoadBalancer :80)	public endpoint; cloud provisions a load balancer, Minikube uses 'minikube tunnel'
Service heart-api-headless	per-pod DNS discovery so Prometheus scrapes every replica
Ingress heart-api.local	alternative HTTP entry (nginx ingress class)
HorizontalPodAutoscaler	2-5 replicas at 70 % CPU (needs metrics-server)
Prometheus / Grafana	monitoring stack with provisioned data source and dashboard (Section 11)

Table 10. Kubernetes resources.

Deploy on Minikube

```
minikube start --driver=docker
minikube addons enable metrics-server ingress
docker build -t heart-disease-api:1.0.0 .
minikube image load heart-disease-api:1.0.0          # no registry needed
kubectl apply -k k8s/
kubectl -n heart-mlops rollout status deploy/heart-api
minikube tunnel                                       # separate terminal: gives the LoadBalancer an IP
kubectl -n heart-mlops get svc heart-api              # EXTERNAL-IP
python scripts/smoke_test.py http://<EXTERNAL-IP>
```

Deploy on a public cloud. Push the CI-built image (`ghcr.io/<owner>/heart-disease-api:<sha>`), set it in `k8s/deployment.yaml` with `imagePullPolicy: Always`, and apply the same manifests to the cluster; the Service of type LoadBalancer yields a public IP.

SCREENSHOT TO ADD: `screenshots/k8s_get_pods.png`
kubectl -n heart-mlops get pods,svc,hpa - pods Running and a LoadBalancer address.

SCREENSHOT TO ADD: `screenshots/k8s_rollout.png`
kubectl rollout status / describe deployment.

SCREENSHOT TO ADD: `screenshots/k8s_predict_via_lb.png`
curl POST /predict through the LoadBalancer / Ingress address.

11. Monitoring and logging

Structured request logging. Every request produces one JSON log line (timestamp, level, request_id, method, path, status, latency in ms, client address) and every prediction produces a second line (prediction, confidence, model_version). The `X-Request-ID` header is honoured or generated and returned, allowing a client report to be traced to log lines. Raw patient features are **not** logged unless `LOG_FEATURES=true`. Output goes to stdout (collected by Docker/Kubernetes) and optionally to `LOG_FILE`.


```
{"ts":"2026-10-20T05:16:12.948Z","level":"INFO","logger":"heart-api","msg":"request","request_id":"8abfa35976b0",
 "method":"POST","path":"/predict","status":200,"latency_ms":10.4,"remote_addr":"127.0.0.1"}
```


Prometheus metrics (`GET /metrics`, a dependency-free implementation of the text exposition format):

Metric	Type	Use
http_requests_total{method,endpoint,status}	counter	traffic and error rate
http_request_duration_seconds{method,endpoint}	histogram	latency percentiles (p50/p95/p99)
heart_inference_seconds	histogram	model inference time
heart_predictions_total{label}	counter	predicted-class mix (input/prediction drift signal)
heart_prediction_confidence	histogram	confidence distribution shift
prediction_errors_total{reason}	counter	validation / bad JSON / inference failures
heart_model_info{version,model}	gauge	which model version is live
process_uptime_seconds	gauge	restarts

Table 11. Exported metrics.

Excerpt scraped from the live local server after the load test (`scripts/load_test.py`, 120 requests including 5 % deliberately invalid):


```
heart_model_info{model="logistic_regression",version="1.0.0-20261020052850"} 1
heart_predictions_total{label="0"} 25
heart_predictions_total{label="1"} 95
http_requests_total{endpoint="/health",method="GET",status="200"} 2
http_requests_total{endpoint="/metrics",method="GET",status="200"} 1
http_requests_total{endpoint="/predict",method="POST",status="200"} 119
http_requests_total{endpoint="/predict",method="POST",status="422"} 6
http_requests_total{endpoint="/ready",method="GET",status="200"} 1
prediction_errors_total{reason="validation"} 6
```


Dashboards and alerts. `docker compose up` starts Prometheus (:9090) and Grafana (:3000, admin/admin) with the data source and the *Heart Disease API - Monitoring* dashboard provisioned automatically (request rate, p95 latency, 5xx ratio, predictions by class, mean confidence, rejected requests, totals, uptime). On Kubernetes the same dashboard is mounted from a ConfigMap and Prometheus discovers all pods through the headless Service. `monitoring/alerts.yml` defines alerts for target down, >5 % 5xx, p95 latency >500 ms and a share of positive predictions above 80 % (possible drift).

SCREENSHOT TO ADD: `screenshots/grafana_dashboard.png`
Grafana dashboard 'Heart Disease API - Monitoring' after running scripts/load_test.py.

SCREENSHOT TO ADD: `screenshots/prometheus_targets.png`
Prometheus - Status > Targets showing heart-api UP.

SCREENSHOT TO ADD: `screenshots/api_logs.png`
kubectl logs / docker logs showing structured JSON request and prediction logs.

12. Architecture overview

Request path. A client sends JSON to the LoadBalancer/Ingress, which routes to one of the API pods. The service validates the payload, runs the same pipeline used in training, returns the prediction with its confidence, and records logs and metrics. Prometheus scrapes each pod; Grafana visualises the metrics.

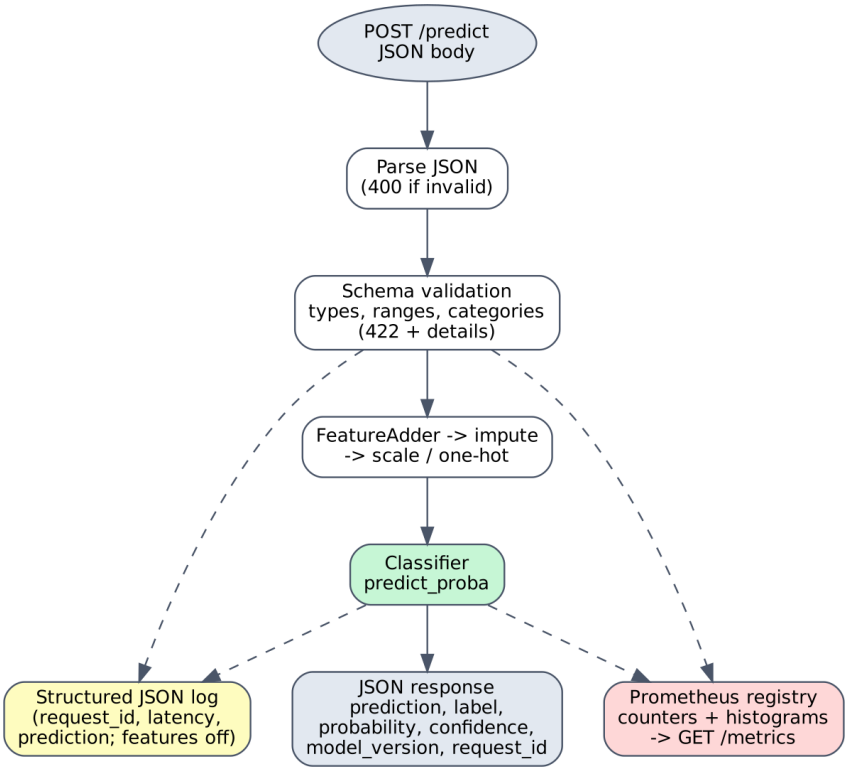


Figure 18. Inside the API: validation, preprocessing, inference, logging and metrics.

Key design decisions

Decision	Rationale	Alternative considered
Single sklearn Pipeline artefact	eliminates train/serve skew; one file to version	separate scaler/encoder pickles
Imputation inside the pipeline	no leakage in CV; API accepts missing ca/thal exactly as training	impute in the cleaning script
Flask + gunicorn (1 worker, threads)	simple, small image; consistent in-process metrics	FastAPI (automatic schema docs); multi-worker with prometheus multiprocess mode
Hand-rolled validation module	framework-free, unit-testable, precise error messages	pydantic models
Dependency-free /metrics	no extra runtime dependency, easy to read and test	prometheus_client
MLflow with local fallback	assignment tracking requirement plus robustness	Weights & Biases
Kustomize for manifests	single `kubectl apply -k`; no templating engine needed	Helm chart
Model baked into the image	immutable, reproducible deployment; image tag = model version	model pulled from registry at start-up

Table 12. Architecture decisions.

13. Results, limitations and ethical considerations

Outcome. The selected Logistic Regression achieves a cross-validated ROC-AUC of 0.895 ± 0.041 and a hold-out ROC-AUC of 0.960 (accuracy 0.885, recall 0.929, specificity 0.848).

Limitations

- Small sample (303 patients) - high variance; no external validation; single-site data from the 1980s.
- Sample skew toward male patients limits generalisation to women; subgroup performance should be audited before any real use.
- The binary label merges all disease grades; the model predicts presence of angiographic disease, not future events.
- The decision threshold (0.5) was not optimised for a clinical cost trade-off.
- Monitoring detects traffic, latency and prediction-mix shifts but not label-based accuracy decay (labels are not available online).
- Grafana credentials in the demo manifests are placeholders; use Kubernetes Secrets, TLS and authentication for any real deployment.

Ethics and safety. This is an educational system and **not a medical device**. Predictions must not be used for diagnosis or treatment decisions. The service never logs patient features by default, validates all inputs, and exposes the model version so every prediction is attributable.

Future work. Data and prediction drift detection (PSI / KS tests), automated retraining and model registry promotion, canary rollouts, TLS and authentication at the Ingress, calibrated thresholds by cost analysis, and fairness metrics by sex and age group.

14. Requirement-to-evidence checklist

Task (marks)	Evidence in the repository / report
1. Data acquisition & EDA (5)	scripts/download_data.py; heartml/data.py; heartml/eda.py; notebooks/01_eda.ipynb; Sections 3-4; reports/figures/eda_*.png
2. Feature engineering & models (8)	heartml/features.py, modeling.py, train.py; 3 models, grid search, repeated CV, accuracy/precision/recall/F1/ROC-AUC; Section 5
3. Experiment tracking (5)	heartml/tracking.py (MLflow); parent + nested runs with params, metrics, plots, models; Section 6; mlruns/
4. Packaging & reproducibility (7)	models/model.joblib + metadata; requirements*.txt; sklearn Pipeline; seeds; Section 7
5. CI/CD & testing (8)	tests/ (48); .github/workflows/ci.yml (lint, test, train, docker, k8s-validate); Section 8
6. Containerisation (5)	Dockerfile, .dockerignore, /predict returns prediction + confidence; scripts/smoke_test.py; Section 9
7. Production deployment (7)	k8s/*.yaml (Deployment, LoadBalancer, Ingress, HPA, kustomization); Section 10
8. Monitoring & logging (3)	JSON logs, /metrics, Prometheus + Grafana (compose and k8s), alerts; Section 11
9. Documentation (2)	README.md and this report: setup, EDA/modelling, tracking, architecture diagrams, CI/CD + deployment screenshots, repository link

Table 13. Mapping of assignment tasks to deliverables.

Production-readiness requirements. (1) Scripts run from a clean setup using the requirements file - yes, via Makefile targets. (2) Model serves correctly in an isolated environment - Dockerfile + smoke test (screenshots in Section 9). (3) Pipeline fails on code or test errors with clear logs - job dependencies, pipefail and non-zero smoke-test exit.

Appendix A - API reference

Method & path	Request	Response
GET /health	-	200 {status: ok}
GET /ready	-	200 {status: ready, model_version}
GET /model-info	-	model name/version, features + descriptions, hold-out and CV metrics, threshold
POST /predict	13-field JSON object; or list; or {instances: [...]} (max 100); ca and thal may be null	200 {prediction, label, probability_disease, confidence, model_version, request_id} or {predictions: [...]}
		400 invalid JSON | 413 batch too large | 422 validation failed (details per instance)
GET /metrics	-	Prometheus text format 0.0.4

Table 14. HTTP API.

Input ranges enforced: age in [1, 120]; trestbps in [50, 260]; chol in [50, 700]; thalach in [40, 250]; oldpeak in [-3.0, 10.0]; ca in [0, 4]; categorical codes: sex in [0, 1]; cp in [1, 2, 3, 4]; fbs in [0, 1]; restecg in [0, 1, 2]; exang in [0, 1]; slope in [1, 2, 3]; thal in [3, 6, 7].

Appendix B - command reference

```
make setup | data | all | test | lint | run | docker-build | docker-run | smoke | compose-up | k8s-deploy | k8s-status |
python scripts/load_test.py http://localhost:8000 --n 300 --rps 15      # generate traffic for Grafana
kubectl -n heart-mlops logs deploy/heart-api --tail=20                  # structured logs
kubectl -n heart-mlops port-forward svc/grafana 3000:3000               # Grafana on k8s
```